Multi-Tenant Database Proxy

Advanced Database Management — Computer Science (A & B), 4th Semester

Project 02 Manual / Languages: C++ or JavaScript

1 Problem Statement

You will build a database proxy that sits in front of a single shared PostgreSQL server and makes it look, to each tenant, as though they have their own isolated database. Many tenants connect to the proxy at the same time; each tenant identifies themselves during the connection handshake; the proxy forwards their queries to PostgreSQL but transparently rewrites them so that each tenant can only see and modify data in their own schema. A tenant running `SELECT * FROM users` sees only their own users, not anyone else's. A tenant running `CREATE TABLE orders (...)` creates a table inside their own schema without disturbing any other tenant. From the tenant's point of view, they have their own private database. From the proxy's point of view, there is one PostgreSQL server with dozens of schemas, and the proxy is routing requests based on who sent them.

This is the design used by real multi-tenant SaaS platforms. Heroku Postgres uses variants of it. Supabase uses it. The "schemas" feature of PostgreSQL itself was designed specifically to support this pattern. It has significant advantages over the alternatives: compared to running one PostgreSQL instance per tenant, it uses far less memory and disk; compared to putting all tenants in a single unsegmented table with a tenant id column, it gives cleaner isolation, better performance (PostgreSQL can optimize queries that only touch one schema), and better security (a query bug cannot accidentally leak data across tenants because the tenant boundary is enforced outside the application code).

The proxy you will build does four things. First, it speaks enough of the protocol to accept client connections, read requests, and forward them to the backend PostgreSQL. Second, it identifies the tenant at connection time using a simple authentication scheme. Third, it maintains a pool of backend PostgreSQL connections per tenant, so that opening a new tenant session does not require establishing a new connection to PostgreSQL every time. Fourth, it enforces per-tenant resource limits, how many queries per second a tenant can run, how many concurrent connections they can hold, so that one misbehaving tenant cannot starve the others. And alongside all of this, it keeps per-tenant usage metrics that could be used for billing in a real system.

You are not implementing the full PostgreSQL wire protocol. That would be a semester's worth of work on its own. Instead, you will build a simple text-based protocol between your client (`tenant-cli`) and your proxy (`tenant-proxy`), and the proxy will speak to the backend PostgreSQL using an existing PostgreSQL client library (`libpq` in C++, `pg` in JavaScript). This is exactly how real database proxies are built when the front-end protocol does not need to match the back-end, you use the backend's official library and focus the implementation effort on the proxy logic instead. The teaching goal is not "write a protocol parser" (you can do that in the Redis and sharding projects); the teaching goal here is *isolation, routing, and resource management.*

2 What the Final Product Looks Like

When your project is complete, you can start a PostgreSQL backend, start your proxy pointing at it, and connect multiple tenant clients that each see their own isolated data. A session looks like this.

```
# terminal 1: postgres running in docker, or locally
$ docker run --rm -p 5432:5432 -e POSTGRES_PASSWORD=secret postgres:16
[postgres] ready to accept connections on port 5432

# terminal 2: the proxy
$ ./tenant-proxy \
    --port 6000 \
    --backend localhost:5432 \
    --backend-user postgres --backend-password secret \
    --backend-db mydb \
    --tenants tenants.conf
[proxy] loaded 3 tenants: acme, globex, initech
```

```
[proxy] ensuring schemas exist on backend...
[proxy]   schema tenant_acme: ok
[proxy]   schema tenant_globex: ok
[proxy]   schema tenant_initech: ok
[proxy] connection pool: 4 connections per tenant, min 1, max 8
[proxy] ready on port 6000

# terminal 3: tenant acme
$ ./tenant-cli --host localhost --port 6000 --tenant acme --api-key acme_k3y
connected as tenant 'acme' (session id 1)

acme> CREATE TABLE users (id INT PRIMARY KEY, name TEXT);
OK (0 rows, 12ms, pool: 1/4 in use)

acme> INSERT INTO users VALUES (1, 'Ayesha'), (2, 'Bilal'), (3, 'Carol');
OK (3 rows, 8ms, pool: 1/4 in use)

acme> SELECT * FROM users;
id | name
---+-------
 1 | Ayesha
 2 | Bilal
 3 | Carol
(3 rows, 6ms, pool: 1/4 in use)

# terminal 4: tenant globex
$ ./tenant-cli --host localhost --port 6000 --tenant globex --api-key globex_k3y
connected as tenant 'globex' (session id 2)

globex> CREATE TABLE users (id INT PRIMARY KEY, email TEXT);
OK (0 rows, 9ms, pool: 1/4 in use)

globex> INSERT INTO users VALUES (100, 'bob@globex.com');
OK (1 rows, 5ms, pool: 1/4 in use)

globex> SELECT * FROM users;
id  | email
----+-----------------
100 | bob@globex.com
(1 rows, 4ms, pool: 1/4 in use)

# ^ note that globex sees only its own user, not acme's three users.
#   acme's table has (id, name); globex's has (id, email). Different
#   schemas, different tables, complete isolation.

acme> SELECT * FROM users;
id | name
---+-------
 1 | Ayesha
 2 | Bilal
 3 | Carol
(3 rows, 6ms, pool: 1/4 in use)

# ^ acme is also unaffected. Neither tenant can see the other.

# what happens if a tenant exceeds its rate limit
acme> \burst 100  # send 100 queries as fast as possible
sent 20 queries successfully...
ERR rate limit exceeded (20 queries per second, retry in 450ms)
sent 20 more queries...
ERR rate limit exceeded ...

acme> \stats
tenant:         acme
session id:     1
queries total:  45
```

```
bytes sent:     2184
bytes recv:     912
pool in use:    1
pool idle:      3
rate limit:     20/sec (current: 18/sec)
last query:     SELECT 1
```

The defining feature of this session is that three different tenants can all have a table called `users` with completely different columns and data, and can query it naturally without any awareness of the others. The proxy is doing the work of routing each tenant's queries to their own schema. No tenant's query ever touches another tenant's data, even by accident, because the SQL rewriter in the proxy guarantees it.

3 Deployment Options

This project has two pieces to deploy: the backend PostgreSQL server and your proxy. You have several options.

3.1 Everything Local

Run PostgreSQL locally (install it natively, or `apt install postgresql`), then run your proxy as a normal process. Connect tenant clients to the proxy. Simplest setup.

3.2 PostgreSQL in Docker, Proxy Local

Run only PostgreSQL in a container: `docker run -rm -p 5432:5432 -e POSTGRES_PASSWORD=secret postgres:16`. Then run your proxy as a normal process pointing at `localhost:5432`. This is the recommended development setup because it avoids installing PostgreSQL on your machine and because you can reset state instantly by restarting the container.

3.3 Everything in Docker Compose

A `docker-compose.yml` with two services: `postgres` and `proxy`. The proxy service builds from a `Dockerfile` in your repository. Clean and reproducible; good for the grader to run on their own machine.

3.4 Postgres Is Always Required

No matter which deployment you choose, PostgreSQL 14 or later must be available as the backend. The proxy is useless without it; the proxy exists specifically to multiplex access to a real backend database. You cannot fake the backend with an in-memory store, because the whole point of the project is that real SQL runs on a real database.

4 Architectural Overview

The proxy has four major components. Keep them strictly separated.

```
          [ tenant clients ]   (many, one per tenant session)
                   |
                   v
        +----------------------+
        |  Client listener     |  accept, authenticate, read queries
        |  (Section 6)         |
        +----------------------+
                   |
                   v
        +----------------------+
        |  Query rewriter      |  prefix tables with tenant schema
        |  (Section 7)         |
        +----------------------+
                   |
                   v
        +----------------------+
        |  Rate limiter        |  per-tenant token bucket
        |  (Section 8)         |
        +----------------------+
                   |
```

```
                v
        +---------------------+
        |  Connection pool    |  pre-warmed libpq connections
        |  (Section 9)        |  per tenant, each bound to the
        |                     |  tenant's search_path
        +---------------------+
                |
                v
          [ PostgreSQL backend ]
```

Every tenant query passes through every component. The client listener is where the tenant shows up and authenticates. The query rewriter makes sure that unqualified table names (like `users`) are interpreted as being in the tenant's schema. The rate limiter decides whether the query is allowed to run right now. The connection pool finds an idle backend connection for the tenant and sends the query. The response flows back through the same path in reverse.

5 The Tenant Configuration

Tenant configuration is static for this project: a file on disk that the proxy reads at startup. The file has one tenant per line in a simple colon-separated format:

```
# tenants.conf
# format: tenant_id:api_key:rate_limit_per_sec:max_connections
acme:acme_k3y:20:8
globex:globex_k3y:50:16
initech:initech_k3y:10:4
```

For each tenant:

- `tenant_id`: a short alphanumeric identifier (lowercase ASCII, no whitespace). Used as the schema name prefix: tenant `acme` gets the schema `tenant_acme`.
- `api_key`: a string the tenant must present to authenticate. In a real system this would be a per-tenant secret issued by an admin; for this project, a plaintext string in the config file is fine.
- `rate_limit_per_sec`: maximum queries per second this tenant may run. Queries over this rate are rejected with an error (see Section 8).
- `max_connections`: maximum concurrent sessions this tenant may hold at once. Connection attempts beyond this limit are rejected.

At startup, the proxy reads the config file, validates each line, and for each tenant issues a `CREATE SCHEMA IF NOT EXISTS tenant_`*`tenant_id`* against the backend. Tenants whose config is malformed cause the proxy to abort at startup with a clear error; there is no dynamic tenant creation.

Dynamic tenant management (reloading the config file, creating tenants at runtime, deleting tenants) is out of scope for the base project but is a bonus.

6 The Client Protocol

The proxy speaks its own simple text protocol to clients (`tenant-cli`). This is *not* the PostgreSQL wire protocol. Using a custom protocol lets you focus the implementation on the proxy logic without having to implement PostgreSQL's binary protocol, which is out of scope.

6.1 The Connection Handshake

When a client connects, the first thing it must send is a `HELLO` line containing the tenant id and the API key:

```
HELLO tenant_id api_key
```

The proxy responds with either `OK session_id` (a new session has been opened and its local id is `session_id`) or `ERR` followed by a reason (bad credentials, tenant at connection limit, unknown tenant, etc.). A client that tries to send any other command before the `HELLO` is disconnected immediately.

6.2 Per-Session Commands

After a successful `HELLO`, the client can send any of the following commands. Each command is a single line terminated by a newline. Responses may span multiple lines.

- `QUERY sql_text` — Execute a SQL query on behalf of this tenant. `sql_text` is the raw SQL (which may contain spaces). The proxy rewrites it, rate-limits it, forwards it through the connection pool, and returns the results as described in Section 6.3.
- `\stats` — Print session and tenant statistics, including pool usage, query count, and bytes transferred.
- `\tables` — List the tables currently in this tenant's schema. A convenience command that is equivalent to running `SELECT tablename FROM pg_tables WHERE schemaname = 'tenant_`*`tenant_id`*`'`.
- `\burst N` — Send `SELECT 1` queries as fast as possible, `N` of them total. Used for testing the rate limiter. The proxy treats each of these as a normal query; the client just emits them quickly.
- `QUIT` — Close the session. The proxy releases any connection it was using back to the pool and closes the client socket.

6.3 Query Response Format

A `QUERY` response takes one of three forms:

Success with no result rows (for `INSERT`, `UPDATE`, `DELETE`, `CREATE`, etc.):

```
OK (N rows, Tms, pool: X/Y in use)
```

Success with result rows (for `SELECT`):

```
col1 | col2 | col3
-----+------+-----
 v11 |  v12 |  v13
 v21 |  v22 |  v23
...
(N rows, Tms, pool: X/Y in use)
```

You do not need perfect column alignment (that is pure polish). A reasonable left-aligned output with pipes and dashes is fine.

Error:

```
ERR reason
```

Errors include SQL errors from the backend (syntax errors, constraint violations), rate limit rejections, and any other failure. In the SQL-error case, the message should be the PostgreSQL error text, minus any leading `ERROR:` prefix.

7 The Query Rewriter

This is the heart of the project. When a tenant sends a query like `SELECT * FROM users`, the proxy must make sure that the `users` referred to is the `users` table in the tenant's own schema, not some other tenant's `users` table, and not PostgreSQL's system-wide `users` catalog. There are two approaches to achieving this, and you should understand both before choosing.

7.1 Approach 1: SET search_path

PostgreSQL has a session variable called `search_path` that tells the query planner which schemas to look in when a query uses an unqualified table name. If you run `SET search_path = tenant_acme`, then `SELECT * FROM users` is interpreted as `SELECT * FROM tenant_acme.users`. This is a clean, idiomatic PostgreSQL mechanism, and it is the right choice for this project.

The trick is that `search_path` is a per-session setting. It is reset whenever the connection is returned to the pool and reused by another tenant. If you give the same backend connection to tenant acme and then to tenant globex without resetting, globex will see acme's tables. This is a serious correctness (and

security) bug.

The fix is to make each pooled connection "belong" to one tenant at a time, and to issue `SET search_path` before handing out the connection to a session. There are two ways to do this:

1. **Per-tenant pools**: the connection pool is actually one pool per tenant. A connection for tenant acme has `search_path = tenant_acme` set on it permanently; it is only ever given to acme sessions. A connection for globex is similarly fixed. This wastes some connections (each tenant reserves a minimum pool size regardless of usage), but it is trivially correct.
2. **Shared pool with reset**: one global pool. Before each query, the proxy executes `SET search_path = tenant_X; SELECT 1; SELECT result_of_actual_query`, either as one round-trip batch or as two separate round-trips. This is more efficient but has a subtle trap: if you forget to issue the `SET` on some code path, queries run with the wrong schema silently.

For this project, use **per-tenant pools** (approach 1a). It is simpler to reason about, simpler to implement, and the small efficiency loss does not matter at the scale you will be operating. Document your choice.

7.2 Approach 2: SQL Parsing and Rewriting

An alternative is to actually parse the SQL the client sent, find every table reference, and rewrite it to prepend the tenant's schema. So `SELECT * FROM users u JOIN orders o ON u.id = o.user_id` becomes `SELECT * FROM tenant_acme.users u JOIN tenant_acme.orders o ON u.id = o.user_id`. This is the "purer" approach because it does not rely on `search_path` side effects.

The problem is that parsing SQL correctly is hard. SQL has dozens of forms of table reference: in `FROM`, in `JOIN`, in subqueries, in CTEs (`WITH` clauses), in `INSERT INTO`, in `UPDATE`, in `DELETE FROM`, in `CREATE TABLE`, and more. Writing a parser that handles every case without breaking legitimate queries is a significant effort, and writing one that handles malicious queries without allowing escape bugs is harder still.

This project uses Approach 1 (SET search_path) as the required baseline. Approach 2 (full SQL rewriting) is available as a bonus. For the base project, your query rewriter's job is very simple: verify that the query does not contain an explicit schema reference (no dots in table names) and reject the query if it does, then forward the query unchanged to the backend on a connection whose `search_path` has been set to the tenant's schema.

7.3 The Forbidden-Schema Check

Because the base project uses `search_path` rather than rewriting, a malicious tenant could try to escape their schema by writing an explicit dotted reference: `SELECT * FROM tenant_globex.users`. This would work if the backend user has access to both schemas, which it does, because the backend user is the same regardless of the tenant.

The proxy must prevent this. The simplest approach is a regex-based check on the incoming SQL: reject any query whose SQL contains any dotted identifier of the form `tenant_X.something`. Also reject any query that contains the keyword `SET search_path` (to prevent the tenant from changing their own schema) and the keyword `SET ROLE` (which would let them impersonate another user). This is a denylist, and denylists are never perfect, but for the scope of this project a denylist is sufficient if it is documented and tested.

A more sophisticated approach using actual SQL parsing is in the bonus. For the base project, the denylist is fine. Your test suite must include test cases that try to escape the schema in various ways and verify they are all rejected.

7.4 Handling `CREATE TABLE` and DDL

When a tenant runs `CREATE TABLE orders (...)`, the `search_path` mechanism causes the table to be created inside the tenant's schema. This is exactly what you want. No special handling required.

A subtle case: `ALTER TABLE`, `DROP TABLE`, and similar DDL statements. All of them respect `search_path` for unqualified references. The denylist check from Section 7.3 catches any attempt to qualify a schema explicitly. So DDL Just Works within the tenant's own schema.

8 The Rate Limiter

Each tenant has a configured rate limit in queries per second. The proxy must reject queries that exceed this rate, and must do so quickly so that a burst from one tenant does not slow down others.

8.1 Token Bucket

Use a token bucket algorithm, the standard way to implement a rate limit. Each tenant has a bucket with a capacity equal to the rate limit (in tokens, where one query consumes one token). Tokens refill at the rate of the rate limit per second. When a query arrives, the proxy tries to take one token from the bucket: if the bucket has at least one, the query proceeds and the token is removed; if not, the query is rejected with an error.

The bucket's current state is stored in the tenant's per-tenant structure: `current_tokens` (a floating-point number, not an integer) and `last_refill_time` (an absolute wall-clock time). Whenever the bucket is accessed:

1. Compute `elapsed` = now − `last_refill_time`.
2. Add `elapsed * rate_limit_per_sec` tokens to `current_tokens`, capped at the bucket capacity.
3. Set `last_refill_time` = now.
4. If `current_tokens` ≥ 1, subtract 1 and accept the query. Otherwise reject with an error that includes the time until the next token is available.

The nice property of this algorithm is that it allows bursts up to the bucket capacity, while maintaining a long-term average rate equal to the refill rate. A tenant who has been idle for 10 seconds and then sends 20 queries in quick succession gets all 20 through (assuming a capacity of 20 or more), because their bucket was at full during the idle period.

8.2 Thread Safety

If the proxy is multi-threaded (one thread per client session, which is the recommended approach), the per-tenant bucket is shared between all sessions belonging to that tenant. Protect it with a mutex. Each query takes the mutex briefly (to check and update tokens), then releases it before doing anything slow.

9 The Connection Pool

A tenant session does not open its own connection to the backend for every query. Instead, the proxy maintains a pool of pre-opened backend connections per tenant, and sessions borrow connections from the pool for the duration of a single query.

9.1 Why a Pool

Opening a new PostgreSQL connection is expensive: it takes a TCP handshake, authentication, a startup message exchange, and the backend spawns a new server process (PostgreSQL uses a process-per-connection model). The cost is typically 10–50 milliseconds per connection. If every query from a tenant opened a new connection, the proxy would be bottlenecked on connection setup instead of query execution.

The pool amortizes this cost by keeping a handful of connections open at all times. Queries grab an idle one, use it for a few milliseconds, and return it. A steady workload can be served by a very small pool because connections are released quickly.

9.2 Pool Parameters

Each tenant's pool has three parameters:

- `min_size`: the minimum number of connections the pool maintains even when idle. Set this to 1 (or 2) so that the first query from a tenant is always fast.
- `initial_size`: the number of connections opened at startup. Set this equal to `min_size`.
- `max_size`: the maximum number of connections the pool may open. Equal to the tenant's `max_connections` from the config file. If a session needs a connection and the pool is empty but under `max_size`, open a new one. If the pool is empty and at `max_size`, the session waits.

Connections in the pool are always bound to the tenant's schema: before being added to the pool, the

proxy issues `SET search_path = tenant_id` on the connection. This is done once, at creation time, and the setting sticks for the lifetime of the connection (which is effectively forever).

9.3 Check-Out, Check-In

The pool exposes two operations:

- `get_connection(tenant_id)`: returns a connection from the tenant's pool. If there is an idle connection, return it immediately. If not and the pool is below `max_size`, open a new connection, `SET search_path` on it, and return it. If the pool is at `max_size`, block the caller until an idle connection is returned.
- `release_connection(conn, tenant_id)`: return a connection to the tenant's pool. If the connection is still healthy, it becomes idle and is available for the next query. If it is broken (the backend dropped it, a query failed in a way that left the connection in a bad state, etc.), discard it and, if the pool is below `min_size`, open a replacement.

For "still healthy," a simple heuristic is: if the last query on the connection returned without throwing an exception, it is healthy. If it threw, discard. A production pool would also ping idle connections periodically to detect silently-broken ones, but for this project the simpler heuristic is fine.

9.4 Concurrency

The pool is shared across sessions and must be thread-safe. Use a mutex around the pool's internal state (the list of idle connections, the count of in-use connections). The mutex is held only briefly during check-out and check-in; the actual query happens without the mutex.

10 The PostgreSQL Client

The proxy talks to the backend PostgreSQL using an existing client library. This is the one place in Project 02 where you are allowed to use a third-party library for the core protocol work, because implementing the PostgreSQL wire protocol is out of scope.

10.1 libpq (C/C++)

For C++ implementations, use `libpq`, the official PostgreSQL C client library. It is included with any PostgreSQL installation and comes with good documentation. The key functions are:

```
#include <libpq-fe.h>

PGconn *conn = PQconnectdb(
    "host=localhost port=5432 user=postgres "
    "password=secret dbname=mydb");

PGresult *res = PQexec(conn, "SELECT 1");
if (PQresultStatus(res) != PGRES_TUPLES_OK) {
    // error: PQerrorMessage(conn)
}
int rows = PQntuples(res);
int cols = PQnfields(res);
const char *value = PQgetvalue(res, row_i, col_i);
PQclear(res);

PQfinish(conn);
```

That is essentially the entire API you need. Use `PQexec` for queries; use `PQntuples`/`PQnfields`/`PQgetvalue` to extract results; use `PQerrorMessage` to get error text. Your build system needs to link against libpq: on Ubuntu, `apt install libpq-dev` and then `-lpq` in your linker flags.

11 Metering

Alongside the rate limiter, the proxy tracks per-tenant usage counters. These are the numbers a real SaaS platform would use for billing. For this project you only need to track them; you do not need to present them in a billing report.

Per tenant, track:

- `queries_total`: total number of successful queries run.
- `queries_rejected`: total number of queries rejected by the rate limiter.
- `bytes_sent`: total number of bytes of SQL text sent to the backend.
- `bytes_received`: total number of bytes of result data received from the backend.
- `cpu_ms`: total wall-clock time spent executing queries (an approximation of backend CPU time; use the time from query send to result receive).

The counters are updated inline with query execution and protected by the same per-tenant mutex used by the rate limiter (or a separate one if you prefer finer-grained locking). The `\stats` command reads and displays these numbers for the calling tenant.

12 Phase 1 — Client Protocol, Authentication, and a Single Tenant

The goal of Phase 1 is a proxy that accepts client connections, authenticates them, forwards queries to the backend, and returns the results. One tenant, no rate limiting, no pool (a new connection per query is fine for now), no query rewriting (use `SET search_path` manually on each query).

Build these components:

- A TCP server that accepts multiple concurrent clients. One thread per client session is the simplest approach.
- The tenant config file loader with the format from Section 5.
- A `HELLO`-based authentication step: verify the tenant id exists in the config and the API key matches. Track the number of open sessions per tenant for the purposes of the `max_connections` check.
- At startup, connect to the backend and issue `CREATE SCHEMA IF NOT EXISTS` for every tenant in the config.
- Per session, open a fresh libpq connection to the backend, issue `SET search_path = tenant_`*`id`* immediately, and then accept `QUERY` commands from the client. For each query, call `PQexec`, serialize the result (either "N rows affected" or a table of rows), and send it back to the client.
- The `tenant-cli` client: connect to the proxy, read commands from stdin, send them, print responses.
- The denylist check on incoming queries (reject anything containing `SET search_path`, `SET ROLE`, or a `tenant_*.something` pattern).

At the end of Phase 1, one tenant should be able to connect, create tables, insert data, and query them. The results should appear in the correct schema on the backend (verify with a separate `psql` connection as the admin user: `SELECT * FROM tenant_acme.users` should show the acme data). Two tenants connecting simultaneously should each see their own data.

13 Phase 2 — Connection Pooling and the Rate Limiter

The goal of Phase 2 is to move from "one connection per query" to a proper pool, and to add the rate limiter.

Build these components:

- The per-tenant connection pool (Section 9). Each pool has `min_size`, `initial_size`, and `max_size`. Connections are pre-warmed at startup. Check-out and check-in are protected by a mutex.
- Integration of the pool into the query path: instead of opening a new connection for each query, take one from the pool, run the query, and return it. Handle the case where the pool is at capacity: block the caller until a connection is available, or (optionally) time out after a few seconds and return an error.
- The token-bucket rate limiter (Section 8). Each tenant has a bucket, refill rate, and capacity. Queries arriving when the bucket is empty are rejected with the "retry in Xms" message.
- The metering counters from Section 11. Update them on every query, including rejected queries.
- The `\stats` command that reports all the counters and the current pool state.
- The `\burst` command in the client, which sends `N` trivial queries as fast as possible, used to exercise the rate limiter.

At the end of Phase 2, a single tenant should run queries through a pool that is small and stable. A `\burst 100` command with a rate limit of 20/sec should succeed on the first 20 queries and then hit

the rate limiter; after a second of waiting, another 20 should succeed. Verify this behavior in your test suite.

14 Phase 3 — Multiple Tenants, Isolation Tests, and Benchmark

The goal of Phase 3 is to bring everything together, verify multi-tenant isolation rigorously, and produce a benchmark.

14.1 Multi-Tenant Testing

Write an isolation test script that:

1. Starts the proxy with at least three tenants.
2. For each tenant, creates a table called `users` with different columns.
3. Inserts different data into each.
4. Verifies that each tenant can only see its own users, and that running `SELECT count(*) FROM users` on tenant A returns the count of A's users, not the sum of everyone's.
5. Attempts to escape the schema by running `SELECT * FROM tenant_B.users` from tenant A's session. Verifies that the query is rejected with an error.
6. Attempts to escape via `SET search_path = tenant_B`. Verifies that the query is rejected.
7. Attempts to escape via `SET ROLE admin`. Verifies rejection.
8. Verifies that a query from tenant A that causes a backend error (syntax error, constraint violation) does not affect tenant B's sessions or leave the pool in a bad state.

14.2 Rate Limit Fairness Test

Write a test that exercises rate limiting across tenants:

1. Start the proxy with tenant A (rate 10/sec) and tenant B (rate 50/sec).
2. From two separate client processes, open sessions for A and B simultaneously.
3. Have A run a `\burst 200` and B run a `\burst 500` at the same time.
4. Measure how many of each tenant's queries succeed per second. Verify that A is limited to roughly 10/sec and B is limited to roughly 50/sec, and that A's burst does not affect B's throughput. The two tenants' limits must be independent.

14.3 Benchmark

Measure three things:

- **Throughput under one tenant**: a single client sending `SELECT 1` as fast as possible (with the rate limit removed for this test, or configured at a very high value). Report queries per second. Expect several thousand per second at minimum; a well-implemented proxy can hit 10,000 to 20,000.
- **Throughput under many tenants**: start 5 tenants, open 2 sessions per tenant, and have each session send queries at 1/10 of the tenant's rate limit. Report aggregate queries per second and 95th/99th percentile latency.
- **Connection pool effectiveness**: run a benchmark where every query opens a new connection (bypass the pool), and compare to a benchmark where the pool is enabled. Report the speedup and the difference in 99th-percentile latency. This quantifies exactly how much work the pool is saving.

The benchmark numbers go in the design document along with a discussion of any surprising results.

15 Deliverables and Submission

15.1 What to Submit

A single zip archive containing:

- The complete source tree of your implementation, including the proxy (`tenant-proxy`), the client (`tenant-cli`), and all test and benchmark scripts.
- A `Makefile`, `CMakeLists.txt`, or `package.json` that builds (or installs dependencies for) the proxy with a single command on a recent Ubuntu LTS.
- A `README.md` with build instructions, instructions for setting up the PostgreSQL backend in each of the deployment options from Section 3, the list of supported client commands, the tenant config file format, and any known limitations.

- A `design.pdf` document of at most 12 pages describing: the four-component architecture; the client protocol and the `HELLO` handshake; the two approaches to schema isolation and why you chose `SET search_path` per tenant; the denylist check and its limitations; the token-bucket rate limiter implementation; the per-tenant connection pool with min/max/initial sizing; the metering counters; and a discussion of the isolation tests, the rate limit fairness test, and the benchmark results.
- A `tests/` directory containing: the isolation test script from Section 14.1, the rate limit fairness test from Section 14.2, and unit tests for the denylist regex and the token-bucket algorithm.
- A `benchmark/` directory with the benchmark scripts and the output of your final run.
- A `tenants.conf` example configuration file with at least three tenants.
- A `docker-compose.yml` showing how to run the backend PostgreSQL and the proxy together, for the grader's convenience.

15.2 Archive Naming

`Group[Number]_Project02_MultiTenant.zip`, for example `Group17_Project02_MultiTenant.zip`.

15.3 Demonstration

Each group will present a live demonstration during the week following the submission deadline. The demonstration consists of three parts. First, a walkthrough of the architecture using the design document. Second, a live startup of a PostgreSQL backend, the proxy, and multiple tenant clients. The instructor/TAs will type arbitrary queries in each tenant session, including attempted isolation escapes ("can you run `SELECT * FROM tenant_X.users` from tenant Y's session? what happens?"), rate-limit tests, and cross-tenant benchmarks. Third, a short oral examination in which each group member will be asked to explain, without notes, a specific component of the implementation, with particular focus on why `SET search_path` on a shared pool is dangerous and how the per-tenant pool design avoids the danger. All group members must be present. Missing the demonstration results in a zero for Phase 3.

16 Bonus Opportunities

Each of the following is independent. Bonus credit is awarded on top of the base grade up to a maximum of 15 percentage points. To claim bonus credit, your design document must include a dedicated section explaining which items you attempted and how you verified them.

16.1 SQL Parser for Safer Rewriting (up to 10%)

Replace the denylist check with actual SQL parsing. Use an existing parser library (for example, `libpg_query` from the pganalyze project, which provides PostgreSQL's own parser as a reusable library; or in Node.js, `node-sql-parser`) to parse the incoming SQL into a syntax tree. Walk the tree to find every table reference. Reject the query if any reference is explicitly schema-qualified; otherwise rewrite the query (or leave it to `search_path`) and forward to the backend. Your test suite must include at least 10 tricky test cases that the denylist would incorrectly handle but the parser handles correctly. Describe the parser library you used and why the denylist was insufficient.

16.2 Dynamic Tenant Management (up to 5%)

Add a small HTTP admin interface (on a separate port, protected by an admin API key) that allows creating and deleting tenants at runtime. Creating a tenant issues the `CREATE SCHEMA` and initializes a new per-tenant pool and rate limiter. Deleting a tenant closes all sessions, drops the connection pool, and (optionally) drops the schema. The admin API should be minimal: `POST /tenants`, `DELETE /tenants/`*`id`*, `GET /tenants`.

16.3 Query Result Streaming (up to 5%)

The base project sends the entire query result as a single response. For large result sets, this is inefficient: the proxy has to buffer the whole result in memory before sending any of it to the client. Implement streaming: as rows arrive from the backend, send them to the client in chunks. The client protocol's response format may need a slight extension to support framing (for example, a length-prefixed chunk format). Measure the memory and latency improvement on a query that returns 100,000 rows.

17 Constraints and Prohibitions

The Following Will Result in Loss of Credit

- Use of any existing database proxy or connection pooler as a component of your implementation. This includes but is not limited to PgBouncer, Pgpool-II, Odyssey, HAProxy with database config, Supabase's supavisor, and ProxySQL.
- Use of any existing multi-tenant SaaS framework or library that provides tenant isolation out of the box.
- Faking the backend with an in-memory store or a file-backed database. PostgreSQL 14 or later must be running as the real backend; the proxy must speak to it via libpq. This is the one project where an external library is allowed for the backend protocol work.
- Using approach 1b from Section 7.1 (shared pool with `SET search_path` before each query) in the base project. The per-tenant pool approach (1a) is required because it is trivially correct. The shared pool with reset is allowed as part of a bonus only if you can describe the invariants that make it safe.
- Skipping the denylist check on incoming queries. Without it, any tenant can read any other tenant's data by writing an explicit dotted reference. This is a correctness bug, not a performance issue, and it will be tested.
- Sharing connection pool state across tenants. Tenant A must not be able to exhaust a pool that tenant B needed; the per-tenant pool isolation must hold.
- Use of Python at any layer, including tests and benchmarks. Tests and scripts must be written in C, C++, shell, or JavaScript.
- Submitting code whose commit history shows large, infrequent commits. Your git history should reflect the incremental development of a proxy of this size.

18 Required Reading

The first two items are required reading before you begin. The remaining items are reference material.

1. PostgreSQL documentation, Chapter 5, Section 5.9 "Schemas." The authoritative reference on how PostgreSQL schemas work and how `search_path` affects unqualified name resolution. Read before Phase 1.
2. PostgreSQL documentation, Chapter 32 "libpq – C Library." The API reference for libpq, which you will use to talk to the backend. Sections 32.1 (database connection control functions) and 32.3 (command execution functions) are the important ones. Read before Phase 1.
3. Heroku Engineering Blog, "Multitenant Postgres." A practical discussion of the schema-per-tenant pattern from one of the largest SaaS platforms built on top of PostgreSQL. Useful background on why the pattern is used and what its limits are.
4. Kleppmann, M. *Designing Data-Intensive Applications*, Chapter 2 ("Data Models and Query Languages"), section on "many-to-many relationships" and the discussion of tenant isolation models. Provides the broader context for the approaches to multi-tenant data.
5. PostgreSQL documentation, Chapter 20 ("Client Authentication"), Sections 20.3 ("Authentication Methods") and 20.4 ("Trust Authentication"). Background on how a real system would handle the API key problem rather than plain text in a config file.
6. PgBouncer documentation, `https://www.pgbouncer.org/usage.html`. PgBouncer is the most popular PostgreSQL connection pooler and its docs describe the different pooling modes (session, transaction, statement). Useful for understanding the tradeoffs you are making with your per-tenant pool design.

Penalty Structure

- Missing documentation: –10%
- Security vulnerabilities: –5% per issue
- Poor code quality: –5% to –15%
- Academic integrity violations: Zero grade

Contact & Support

Teaching Assistants:
Jalal Ahmed: `bscs23134@itu.edu.pk`
Khadijah Farooqi: `bscs23128@itu.edu.pk`

Office Hours: Monday & Thursday, 11:30 AM – 2:30 PM